

Trabalho Prático — Paradigmas de Programação

Tema: Introdução ao Rust — Segurança de Memória sem Garbage Collector

Visão geral

- **Equipe:** 7 integrantes
- **Objetivo:** pesquisa aplicada sobre Rust + protótipo funcional que demonstre os conceitos na prática
- **Peso:** até 4,0 pontos (2ª unidade) — nota coletiva + individual
- **Formato:** repositório GitHub público + apresentação oral (12–15 min + até 5 min de perguntas)
- **Prazo do repositório:** não especificado no enunciado — confirmar com o professor

Entregáveis obrigatórios

1. Repositório GitHub (público)

- [] README com nome da UFCA, disciplina, nomes completos dos 7 integrantes e professor
- [] Documentação técnica majoritariamente em Markdown, organizada e com hyperlinks
- [] Código-fonte de todos os exemplos práticos
- [] Instruções completas de instalação, configuração e execução
- [] Seção de referências bibliográficas
- [] Histórico de commits distribuído ao longo do tempo (commits regulares — não vale um único commit no fim)

2. Conteúdo obrigatório da documentação

- [] Pesquisa bibliográfica sobre Rust
- [] Contextualização histórica: por que Rust surgiu e quais problemas de C/C++ ela resolve
- [] Conceitos e mecanismos: ownership, borrowing, lifetimes
- [] Comparação prática com C: vantagens, limitações, cenários de uso, e quais erros são prevenidos automaticamente pelo compilador
- [] Interoperabilidade com Python — opcional pro nosso tema, só se sobrar tempo
- [] Análise crítica: maturidade, ecossistema, comunidade, perspectivas de adoção, limitações

3. Apresentação oral

- [] 12 a 15 minutos + até 5 minutos de perguntas
- [] Todos os 7 integrantes presentes, participando de forma equilibrada
- [] Cada integrante precisa dominar o conteúdo completo — a nota individual depende disso, inclusive nas perguntas

Exemplos sugeridos para a comparação Rust × C

Casos que o compilador do Rust rejeita em tempo de compilação (não apenas em runtime):

- Use-after-free
- Double free
- Dangling reference
- Data race — ex: condição de corrida em C com threads vs. prevenção em tempo de compilação no Rust (ownership + Send/Sync)

Ponto para a análise crítica

As garantias de segurança valem para o "safe Rust" — blocos **unsafe** devolvem a responsabilidade de segurança de memória ao programador. Bom contraponto pra discutir limitações da abordagem.

Divisão de trabalho sugerida (7 pessoas)

- **Contexto e fundamentos (2 pessoas)** — pesquisa bibliográfica, histórico, ownership/borrowing/lifetimes
- **Exemplos práticos (2–3 pessoas)** — implementação dos pares Rust/C (use-after-free, double free, dangling reference, data race)
- **Análise crítica e ecossistema (1–2 pessoas)** — maturidade, comunidade, adoção, limitações do unsafe, e o exemplo opcional de Python se sobrar tempo
- **Documentação e repositório (1 pessoa, com apoio de todos)** — README, organização do Markdown, instruções, referências

Recomendado: revisão cruzada entre frentes e ensaio da apresentação completa por todos (não só da própria parte).